Listing 8.18 Query that returns a single city using the name as the criteria

```
public async Task<City> GetByNameAsync(string name)
{
    return await _context.Cities
        .Include(c => c.Country)
        .Include(c => c.Properties.Where(p => p.AvailableFrom <
        ➥ DateTime.Now))
        .SingleOrDefaultAsync(c => c.Name == name);
}
```

The query differs from the previous one only in the method used to cause execution. This time, you use the `SingleOrDefaultAsync` method. This method expects there to be zero or one matching records in the database. If no records match, the method returns the default value, which is `null` in this case. If more than one record matches the criteria, an exception will be raised. If you expect there to be only one matching record, you can use the `SingleAsync` method, which returns an exception in the event of no matches. If you believe more than one record might match the criteria, you should use the `FirstAsync` or `FirstOrDefaultAsync` method, depending on whether there is a possibility of no matches. This will return the first match based on the default order of the database or the order you specify via an `OrderBy` method. For example:

```
return _context.Cities.OrderBy(c => c.Name).FirstAsync(c => c.Name == name);
```

You use the `SingleOrDefaultAsync` method here because you don’t have total control over the value being passed in to the method. You may think you do because your code generates the links from data that comes from the database. However, you are exposing that data to the outside world when you include it as part of a URL and cannot trust it will not be changed, innocently or otherwise. Next you need to change the code for the